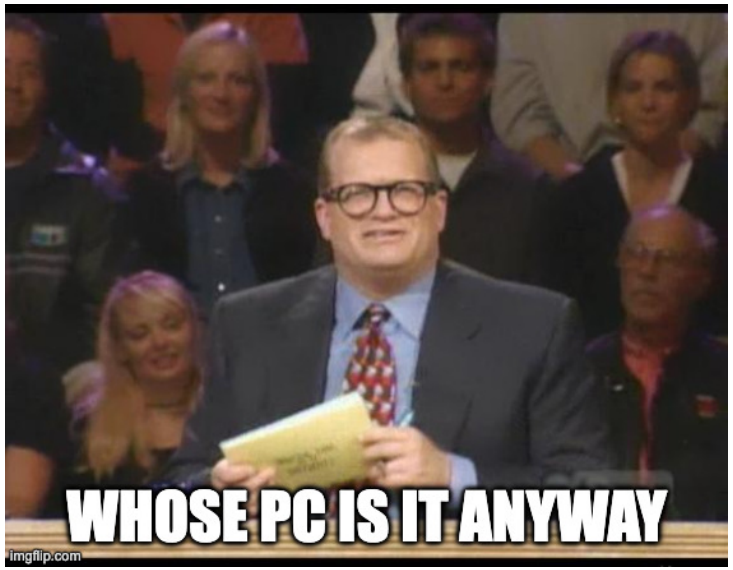
Lecture 33 — Trusted Computing

Jeff Zarnett
jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering
University of Waterloo

September 7, 2026

Trusted Computing



Previously: online multiplayer games use kernel-level anti-cheat mechanisms.

In 2025, for example, Activision/Blizzard announced that they would require TPM 2.0 and Secure Boot to play Call of Duty.

These two are hardware-based functionality that are ostensibly about security.

The History Lesson

The focus on hardware-level protection of what is allowed to run on the computer started around the year 2000.

That year: the publication of the version 1.0 specification of Trusted Computing.

It is *not at all* a coincidence that this conversation started at a time where the hot topic amongst the megacorporations was “stopping music piracy”.

The History Lesson

In June of 1999, the application Napster launched: Peer-to-Peer music sharing.

DRM

The big players looked to stop this through Digital Rights Management (DRM).

DRM: The ability to stop you from doing things they do not want you to do with the media that you purchased.

It is really *digital restrictions management*.

These restrictions only work if the software understands what you are and are not allowed to do, and enforces those constraints.

You can very likely think of some workarounds for these restrictions immediately, if you control the computer and its software.

Legal Disclaimer

Circumvention of DRM restrictions is a legal minefield!

There exist valid reasons and circumstances where it is your legal right to do so, and other circumstances where the same actions are a violation of copyright law.

Consult a lawyer to learn the specifics.

DRM Circumvention

Edit the binary to change the if-else condition?

Find the encryption key in process memory?

Redirect network call?

Load your own kernel module?

Arms Race

The key factor in all of the examples given above?

The person who purchased and administers the computer has a lot of ability to modify application software and even the OS.

They’re not trivial, but once is enough.

Arms Race

The prevention of circumvention of DRM has to move down, into hardware.

That's the only way to break that condition where the administrator of the computer has enough control to bypass restrictions.

TC Quote

TC provides a computing platform on which you can't tamper with the application software, and where these applications can communicate securely with their authors and with each other. The original motivation was digital rights management (DRM): Disney will be able to sell you DVDs that will decrypt and run on a TC platform, but which you won't be able to copy. The music industry will be able to sell you music downloads that you won't be able to swap. They will be able to sell you CDs that you'll only be able to play three times, or only on your birthday.

– Ross Anderson

And Then, It Got Worse

The current state of DRM is that it allows the corporation to alter the deal at any time.

In 2026, for example, Sony deleted movies (including T2!) that customers paid for.

Yes, yes, licensing vs ownership. But they *can* do this.

Hardware Solution

If I control the kernel – including live-patching it – I can bypass restrictions.

What if the computer refuses to boot a kernel that is not “approved”?

… And such an approved kernel would forbid any live-patching!

(People will find a way.)

Not Just Anti-Piracy

This can go beyond preventing people from watching media or copying software.

It can be used to delete content that the government, or other powerful actors, don't want you to have.

The standard dystopian example of this: imagine a journalist has published an exposé on the corruption and criminality of the dictator of a country.

What if the dictator simply has the ability to order that all copies of that document are deleted from everyone's computer?

Not Useless

At least at present, if verification fails, the computer is not useless…

It's just in an untrusted state.

You can do a lot… but not play the desired game or consume restricted media.

More Useless

It is, however, not unthinkable that the future state is that it will be difficult to run the computer in an untrusted state at all.

As life becomes increasingly conducted online, it gets harder over time to use a “free” computer if you cannot use it to do the things needed in everyday life.

Examples: online banking, pay tax, order online, edit documents…

It is also important to note that perfection on digital restrictions like this is impossible for multiple reasons.

One of them is referred to as “the analog hole”.

People will find a way; it’s just a question of how hard it is.

Say No to Brain Implants, Kids

Under no circumstances should you let anybody put a chip in your brain to "permit" you to watch restricted content.

The future where media companies are inside your brain is corporate serfdom, not Cyberpunk 2077.

Alternative Uses

Gaming provides a legitimate example.

The kernel-level anti-cheat module can be backstopped by the trusted computing platform.

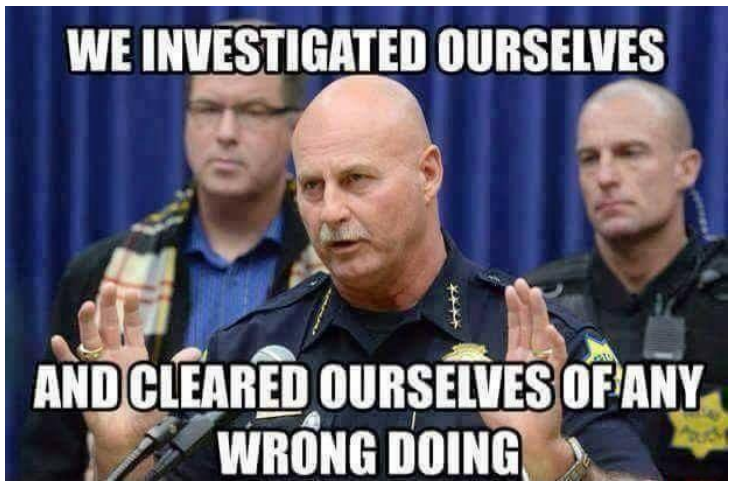
War Thunder

There are also scenarios where locking down documents may serve reasonable government or military purposes.

Kernel Compromise

If malware is installed in the system firmware or boot loader, the malware can survive if the OS is reinstalled.

The OS cannot find it, because the OS is running itself on a compromised foundation.



This is a bootkit.

Secure Boot

The supposed solution to that is called Secure Boot.

It is a security feature in the UEFI standard to verify these low-level foundational components to be sure they are valid.

Verify first, and then transfer control.

Nothing's Perfect

There are numerous examples of bypassing secure boot.

Example: BlackLotus from 2022.

This malware uses a known vulnerability to bypass secure boot, and exactly like the driver revocation problem we discussed.

Access Denied

Alongside the idea of secure boot is measured boot.

Instead of just allow or deny kind of semantics, measured boot hashes and records the state of the things that ran, whether that's been altered or not.

Change detection!

Measured Boot

Arch Linux: these measured boot values can also be used as a gating function for whether access is permitted.

We need some hardware support: the Trusted Platform Module (TPM)

The TPM permits management of encryption keys.

TPM Example: Encrypt Drive

Suppose the boot drive of the system is encrypted.

The encryption key will be sealed by the hardware TPM.

The module will release the encryption key only if the assessment of the boot state is acceptable.

But it also needs to handle change!

Attestation

We've discussed two ways in which the state of the system can be assessed.

Do those checks produce reliable results?

Attestation

The solution is called attestation: verifying, externally, the claims.

Attestation has a few steps:

1. Measurement: collect evidence.
2. Request verification from remote system.
3. Challenger/appraiser sends random one-time code.
4. Combine this code with measurement.
5. Use TPM to sign using TPM signing key.
6. Send to challenger and verify.
7. If signature, one-time code, and measurement are valid: pass.

Attestation

For the game, inspect the state of the system.
    Pass means you can launch the game.

Probably there are periodic re-checks? Kick if failed?

Attestation

Get around the problem of self-report; external validation.

The validator is not under the control of the bad actors.

We should view the threat model from the view of the game server.

Attestation

The one-time code prevents replaying an old response.

The cryptographic signing is hard to forge because only the machine's TPM has the (unique) signing key.

Validating the signing key is farther down the cryptography and security track than we consider to be in-scope for this course.

Attestation

We forgot to talk about one thing, though: how can we be sure that the measurement is valid?

Remember that verification happens at each stage, before we go on to the next.

If we start from a secure place, like the UEFI, it will accurately record the boot loader's measurement.

If the kernel is compromised, that compromise has already been recorded before it gets a chance to run.

And the record cannot be rewritten by a malicious kernel.

Rewrite the Record?

That just moves the problem one step along: why can the kernel not rewrite the record?

The TPM does not offer that functionality!

Read, Reset, Extend.

Chain of Trust

All of this might make you feel a certain level of concern as to what exactly is going on in the devices that you allegedly own.

There are so many points at which a bad actor could compromise the security.

Chain of Trust

Part of the argument for open-source software is that you can, at least in principle, audit the code in it.

Closed source can be audited too, with more difficulty.

But it is a huge undertaking!

The problem is worse in the world of cloud computing.

Cloud computing is rather diametrically opposed to that view: you own nothing, but instead rent some resources from the cloud provider.

So you need to trust the cloud provider, its hardware, its hypervisor, or the people who run the cloud computing platform in general.

Near-Miss

Early in 2024, a compromise was discovered in the `xz` library.

The library is used for data compression; some versions of `openssh` rely on it.

This is a strong example of what's termed a "supply chain attack".

Near-Miss

Almost any modern piece of software that isn't a toy or academic assignment is built in a compositional way.

A uses libraries *B*, *C*, *D* each of which has dependencies *E*, *F*, *G*, *H*...

Compromising any one of the dependencies *F* could result in a vulnerability in *A*.

But if *A* is `openssh`, you know, the *secure* shell daemon...

Near-Miss

And yet, it's worse than that.

The compromise of the library was in the build system for `xz`, not the code itself.

Merely examining the source of the library wouldn't have found the problem.

Bad Things or Bad Actors

Ken Thompson told us in a 1984 lecture that you cannot trust code that you did not write yourself or that's built using a toolchain that you did not write.

We have to put some trust in *some people*, otherwise we will never be able to accomplish anything.

Bad Things or Bad Actors

It feels bad, in a way, to say that we have to just trust other people.
After all: *we live in a society.*

Obviously, a battery-powered drill won't leak your health records to hackers...

You're still counting on it to not explode in your hands or burn your home down.

Trust is fundamental to society. Look skeptically at software you did not write.
Do so proportionally with the magnitude of risks.